Ranqhay: Sistema Web de Gestión de Ventas

Planteamiento de un proyecto de sistema web · Tarea 05

Proyecto	Ranqhay — Sistema Web de Gestión de Ventas para pequeños negocios de cualquier rubro
Curso	Fullstack Developer Software
Integrantes	Int. 1: José Manuel Meza Mesahuanca — Coordinador del proyecto y Responsable de documentación Int. 2: Roni Joseph Qqueso Meléndez — Analista del problema y Responsable de planificación Int. 3:Elisabet Florez Vargas — Analista de requisitos
Fecha	16 de octubre de 2026

1.Enunciado de la tarea

El equipo propone **Ranqhay**, un sistema web que registra cada venta al momento, descuenta el stock, controla la caja diaria y entrega reportes, para pequeños negocios que hoy llevan sus ventas a mano.

Aspecto	Propuesta del equipo
Organización destinataria	Pequeños negocios minoristas del Perú, de cualquier rubro (bodegas, minimarkets, ferreterías, librerías, tiendas de ropa o repuestos): un local, 2 a 10 trabajadores. Se validará con un negocio piloto real.
Necesidad	Controlar ventas, stock y caja de forma confiable y en tiempo real.
Solución	Aplicación web de 3 capas: frontend React, API REST en Node.js y base de datos PostgreSQL (Supabase), publicada en HostArmada con dominio propio y SSL.
Usuarios	Administrador (dueño o encargado) y Cajero.
Esta entrega	Propuesta conceptual: problema, justificación, objetivos, alcance, requisitos y plan. No incluye código; el anexo deja lista la guía para la etapa de construcción.

Descripción del proyecto

Se usa desde el navegador de la PC de caja o del celular, sin instalar nada, y reúne cinco áreas:

- **Punto de venta:** venta, total, vuelto y ticket.
- **Productos e inventario:** catálogo, entradas de mercadería y alertas de stock mínimo.
- **Clientes:** registro opcional del comprador.
- **Caja:** apertura y cierre diario con diferencia.
- **Reportes:** ventas por día, producto y vendedor, y panel del día.

El frontend envía cada operación como petición HTTP con JSON a la API REST, que es la única que lee y escribe en la base de datos.

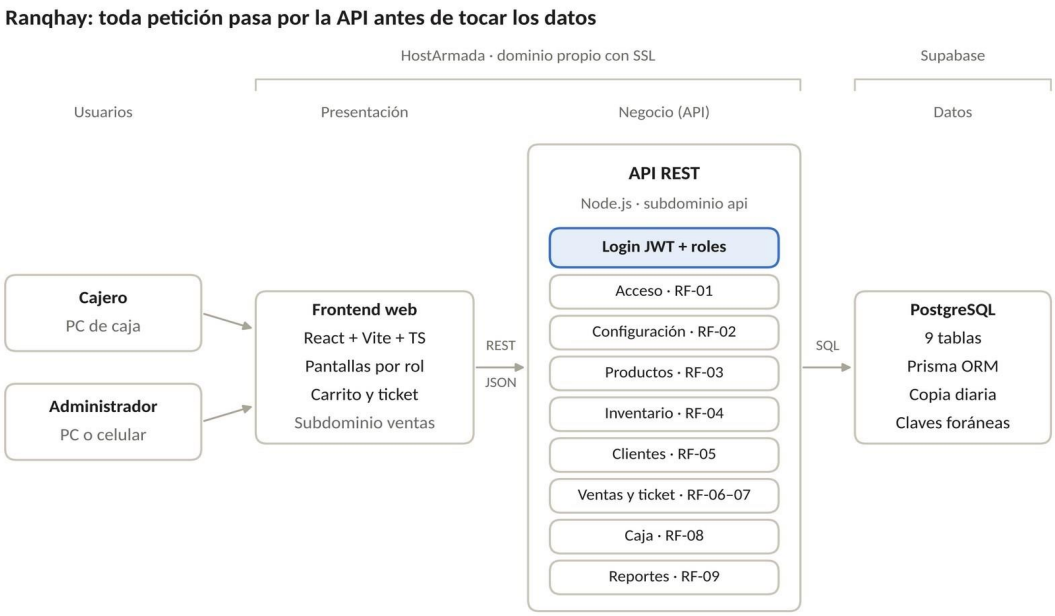


Figura 1. Arquitectura de Ranqhay: 3 capas y 8 módulos de la API.

Usuario	Quién es	Qué necesita del sistema
Administrador	Dueño o encargado	Configurar el negocio, mantener productos y precios, registrar mercadería, gestionar usuarios, revisar la caja y ver reportes desde cualquier lugar.
Cajero	Vendedor o personal de caja	Registrar ventas rápido, consultar precio y stock, cobrar, dar vuelto exacto, abrir y cerrar su caja.

Beneficiarios indirectos: los clientes del negocio (atención más rápida, precios correctos y comprobante).

2. Objetivo de aprendizaje

Con esta propuesta el equipo formula un sistema web completo a nivel conceptual. Cada capacidad del objetivo de aprendizaje se evidencia así:

Capacidad	Cómo se evidencia en Ranqhay	Sección
Identificar y justificar una problemática	Control manual y disperso de ventas, stock y caja; 6 causas con sus consecuencias y beneficios esperados	3, 4.1, 4.2
Definir objetivos y alcance	1 objetivo general, 5 específicos, 8 módulos incluidos y 6 exclusiones	4.3
Especificar requisitos	9 requisitos funcionales (RF-01 a RF-09) y 7 no funcionales (RNF-01 a RNF-07), con prioridad y forma de verificar	4.4
Elaborar un plan de trabajo	14 actividades, responsables y duraciones; cronograma de 10 semanas con 5 hitos	4.5, 7
Organización y colaboración	5 roles repartidos entre 3 integrantes; cada uno redacta y todos revisan	6

3. Situación problemática y reto

Problemática elegida: muchos pequeños negocios no saben con exactitud cuánto venden, cuánto stock tienen ni por qué la caja no cuadra, porque su control es **manual y disperso**. Parte de la idea de referencia «Sistema de inventario» y la amplía con ventas, caja y reportes.

Situación actual

- Cada venta se cobra con una lista de precios impresa y se anota en un cuaderno, o no se anota cuando hay cola.
- Al cierre, el dueño suma las anotaciones y las compara con el efectivo.
- El stock se controla a simple vista y las compras se coordinan por WhatsApp; Excel solo cuando hay tiempo.

Causas y consecuencias

Causa	Consecuencia	Afectados
Ventas anotadas a mano o sin anotar	La caja no cuadra y no se sabe dónde está la diferencia	Dueño y cajeros
Stock controlado «a ojo»	Productos agotados sin aviso y productos vencidos por comprar de más	Dueño y clientes
Listas de precios impresas desactualizadas	Cobros equivocados, reclamos y pérdida de margen	Cajeros y clientes
Totales y vuelto con calculadora	Colas en horas punta y errores de vuelto	Cajeros y clientes
Información dispersa (cuaderno, Excel, WhatsApp)	Compras por intuición, sin saber qué se vende más ni la ganancia real	Dueño
Sin respaldo de registros	Si el cuaderno se pierde, se pierde el historial	Dueño

Delimitación: comercios minoristas de un solo local, con venta presencial en mostrador y 1 o 2 personas atendiendo a la vez.

Reto: ¿cómo pueden los pequeños negocios controlar sus ventas, su inventario y su caja de forma confiable y en tiempo real, sin depender del cuaderno?

4. Instrucciones para el desarrollo

Resultado de las seis actividades indicadas en la guía, aplicadas a Ranqhay.

4.1 Identificar el problema

- **Dificultades:** descuadres de caja, productos agotados o vencidos, precios equivocados, colas y decisiones sin datos.
- **Personas afectadas:** el dueño decide y asume las pérdidas sin datos confiables; los cajeros cargan con los descuadres y la presión en horas punta; los clientes sufren colas y errores de cobro.
- **Consecuencias de no resolverlo:** el negocio pierde dinero de forma invisible (descuadres, mermas y ventas perdidas) y no puede crecer, porque el control manual ya está al límite.

Necesidad identificada: una herramienta centralizada que registre cada venta al momento, mantenga el stock actualizado, cierre la caja con exactitud y muestre reportes confiables.

4.2 Justificar la propuesta

Importancia: las ventas son la fuente de ingresos del negocio. Un registro digital convierte cada venta en un dato que actualiza a la vez el stock, la caja y los reportes.

Problema	Beneficio esperado	Cómo lo logra el sistema
La caja no cuadra	Cierre exacto y diferencias explicables	Cada venta queda registrada con su método de pago
Productos agotados o vencidos	Reposición a tiempo, menos mermas	El stock baja solo y avisa al llegar al mínimo
Precios equivocados	Un precio único y vigente	Catálogo central para todos los usuarios

Colas y errores de vuelto	Atención más rápida	Búsqueda o código de barras; total y vuelto automáticos
Problema	**Beneficio esperado**	**Cómo lo logra el sistema**
Compras por intuición	Decisiones con datos	Reportes de más vendidos, por día y por vendedor
Riesgo de perder el historial	Información respaldada	Base de datos con copia diaria

Por qué una solución web:

- **Acceso desde cualquier lugar:** el dueño revisa las ventas desde su celular.
- **Sin instalación:** funciona en el navegador; si la PC de caja falla, se usa otra.
- **Varios usuarios a la vez** sobre los mismos datos.
- **API REST reutilizable:** servirá luego para una app móvil o la facturación electrónica.
- **Bajo costo:** tecnologías libres del curso, hosting propio y plan gratuito de Supabase.

4.3 Definir los objetivos y el alcance

Objetivo general: desarrollar una aplicación web de gestión de ventas para pequeños negocios de cualquier rubro, con un frontend web conectado mediante una API REST a una base de datos PostgreSQL, que registre las ventas en tiempo real, controle el inventario y la caja diaria y genere reportes, para reducir errores de cobro y descuadres y mejorar las decisiones del propietario.

Código	Objetivo específico	Problema que atiende
OE-01	Implementar un punto de venta que registre cada venta con sus productos, total y método de pago, y emita un ticket.	Ventas sin registro, colas y errores de vuelto
OE-02	Automatizar el inventario: stock actualizado con cada venta y entrada, con alertas de stock mínimo.	Productos agotados o vencidos
OE-03	Controlar la apertura y el cierre de caja comparando el efectivo esperado con el contado.	Descuadres de caja
OE-04	Generar reportes de ventas por período, producto y vendedor.	Compras por intuición
OE-05	Restringir cada función según el rol (administrador o cajero) y respaldar la información.	Información sin control ni respaldo

Usuarios principales: Administrador y Cajero. **Versión 1:** venta presencial de un solo local, con estos 8 módulos:

Módulo	Qué incluye	Lo usa	RF
Acceso y usuarios	Inicio de sesión, roles, alta y baja de usuarios	Todos / Admin.	RF-01
Configuración	Datos del negocio, moneda, impuesto y unidades de medida	Admin.	RF-02
Productos	Registro, edición, búsqueda y categorías	Admin.; cajero consulta	RF-03
Inventario	Entradas de mercadería, stock automático, alertas	Admin.	RF-04
Clientes	DNI/RUC, nombre, teléfono; «cliente general»	Cajero y admin.	RF-05
Ventas	Carrito, total, vuelto, método de pago, ticket y anulación	Cajero; admin. anula	RF-06, RF-07
Caja	Apertura con monto inicial y cierre con diferencia	Cajero; admin. revisa	RF-08

Reportes y panel	Historial, reportes por fecha, producto y vendedor, PDF/Excel, panel del día	Admin.	RF-09

Fuera del alcance:

- Facturación electrónica ante la SUNAT (el ticket es un comprobante interno).
- Tienda en línea, pagos en línea y delivery (tarjeta y billetera se registran, no se procesan).
- Funciones de un solo rubro: mesas de restaurante, lotes de farmacia, tallas y colores.
- App móvil nativa (será web adaptable al celular).
- Varios locales o almacenes; órdenes de compra, contabilidad y planillas.
- Funcionamiento sin internet.

Supuestos: el negocio tiene una PC con navegador e internet; el lector de código de barras USB y la impresora de tickets son opcionales (el ticket también sale en PDF).

4.4 Identificar los requisitos

Requisitos funcionales. Prioridad **Alta** = indispensable para vender; se construye primero.

Código	Módulo	Requisito funcional	Prioridad
RF-01	Acceso	El sistema permitirá iniciar sesión con usuario y contraseña, mostrará solo las opciones del rol, y permitirá al administrador registrar, editar y desactivar usuarios (nombre, usuario, contraseña y rol).	Alta
RF-02	Configuración	El sistema permitirá al administrador configurar los datos del negocio (nombre, RUC, dirección, teléfono y logo), la moneda, el % de impuesto y las unidades de medida, que se usarán en tickets y reportes.	Alta
RF-03	Productos	El sistema permitirá al administrador registrar, editar y desactivar productos (código o código de barras, nombre, categoría, unidad, precio de compra y de venta, stock inicial y mínimo) y categorías; cualquier usuario podrá buscarlos por nombre, código o categoría.	Alta
RF-04	Inventario	El sistema permitirá registrar entradas de mercadería (producto, cantidad, costo, proveedor y fecha), aumentará el stock automáticamente y mostrará una alerta cuando el stock sea igual o menor al mínimo.	Alta
RF-05	Clientes	El sistema permitirá registrar clientes (DNI o RUC, nombre y teléfono) y asociarlos a una venta; si no se elige uno, se usará «cliente general».	Baja
RF-06	Ventas	El sistema permitirá al cajero registrar una venta por búsqueda o código de barras, calculará subtotal, descuento, impuesto, total y vuelto, registrará el método de pago (efectivo, tarjeta, Yape o Plin), descontará el stock y no permitirá vender más unidades que las disponibles.	Alta
RF-07	Ventas	El sistema generará un ticket (datos del negocio, número correlativo, fecha y hora, productos, impuesto, total, pago y cajero) para imprimir o descargar en PDF, y permitirá al administrador anular una venta con motivo, reponiendo el stock.	Alta
RF-08	Caja	El sistema permitirá al cajero abrir su caja con un monto inicial y cerrarla con el efectivo contado, mostrando el monto esperado y la diferencia.	Alta
RF-09	Reportes	El sistema permitirá al administrador consultar el historial de ventas con filtros (fechas, cajero, cliente, ticket), generar reportes por día, producto y vendedor exportables a PDF y Excel, y ver un panel con lo vendido hoy, el número de ventas, los 5 más vendidos y las alertas de stock.	Media

Ejemplo de verificación (RF-06): con stock 10, vender 3 unidades deja 7; intentar vender 8 debe ser rechazado.

Requisitos no funcionales.

Código	Categoría	Requisito no funcional	Cómo se verifica
RNF-01	Seguridad	El sistema deberá guardar las contraseñas con hash (bcrypt), restringir cada función según el rol, cerrar la sesión tras 30 min sin uso y funcionar solo con HTTPS.	Ninguna contraseña legible en la BD; un cajero en la URL de reportes recibe «acceso denegado»; http:// redirige a https://
Código	**Categoría**	**Requisito no funcional**	**Cómo se verifica**
RNF-02	Rendimiento	El sistema deberá confirmar una venta en ≤ 2 s y mostrar búsquedas en ≤ 1 s, con 5 usuarios simultáneos y 5 000 productos.	Medir 10 ventas de prueba con las herramientas del navegador
RNF-03	Usabilidad	Un cajero nuevo deberá registrar una venta de 3 productos en ≤ 1 min tras 30 min de capacitación.	Prueba cronometrada con un usuario real
RNF-04	Compatibilidad	El sistema deberá funcionar en las 2 últimas versiones de Chrome, Edge y Firefox, en pantallas de 360 px a 1920 px.	Probar las pantallas principales en cada navegador y en un celular
RNF-05	Integridad	Cada venta se guardará como una transacción completa en PostgreSQL 15+, con claves primarias y foráneas.	Forzar un error al vender: no queda nada a medias
RNF-06	Disponibilidad	El sistema deberá hacer una copia de seguridad diaria y conservar las de los últimos 7 días.	Revisar el registro de copias y restaurar una en prueba
RNF-07	Arquitectura y mantenibilidad	El frontend se comunicará solo por una API REST con JSON, documentada en Swagger; el código estará en Git con README y sin claves (variables de entorno).	Probar endpoints con Postman y /api/docs; instalar el sistema siguiendo solo el README

Las cifras son metas para un pequeño negocio y pueden ajustarse tras la entrevista con el negocio piloto.

4.5 Planificar el proyecto

6 fases, 14 actividades, 10 semanas (12–15 h semanales por integrante). La construcción va en **3 sprints**: cada uno entrega módulos funcionando, así siempre existe un sistema usable con los requisitos de prioridad alta. La semana 1 empieza cuando el docente apruebe la propuesta.

ID	Fase	Actividad	Resp.	Depende	Duración	Entregable
A0	Gestión	Coordinación y reunión semanal de seguimiento	Int. 1	—	Sem. 1–10	Acta semanal
A1	Análisis	Entrevista y observación en el negocio piloto	Int. 2	—	3 días	Notas y datos reales
A2	Análisis	Validar requisitos y redactar historias de usuario	Int. 3	A1	5 días	Requisitos aprobados
A3	Diseño	Modelo de datos PostgreSQL (diagrama E-R)	Int. 3	A2	4 días	Diagrama E-R y script
A4	Diseño	Prototipos de pantallas (wireframes)	Int. 2	A2	4 días	Prototipo navegable
A5	Diseño	Contrato de la API REST, repositorio, Supabase y hosting	Int. 1	A3	3 días	Entorno listo
A6	Desarrollo	Sprint 1: acceso, configuración y productos (RF-01 a RF-03)	Todos	A4, A5	2 sem.	Módulos base funcionando

A7	Desarrollo	Sprint 2: inventario, clientes, venta y ticket (RF-04 a RF-07)	Todos	A6	2 sem.	Venta completa
A8	Desarrollo	Sprint 3: caja y reportes (RF-08 y RF-09)	Todos	A7	1 sem.	Sistema completo
A9	Pruebas	Pruebas de API y pantallas por cada RF y RNF; correcciones	Int. 3	A8	3 días	Informe de pruebas
A10	Pruebas	Prueba de aceptación con el dueño y un cajero	Int. 2	A9	2 días	Acta de conformidad
A11	Despliegue	Publicar en HostArmada, conectar Supabase, cargar productos	Int. 1	A10	2 días	Sistema en línea (HTTPS)
A12	Despliegue	Capacitación y manual de usuario	Int. 2	A11	2 días	Manual de usuario
ID	**Fase**	**Actividad**	**Resp.**	**Depende**	**Duración**	**Entregable**
A13	Cierre	Documentación técnica final y presentación	Int. 1	A11	3 días	Informe final

Reparto en los sprints: Int. 1 construye la API REST; Int. 2, las pantallas del frontend; Int. 3, las tablas en PostgreSQL, los reportes y las pruebas. Un requisito está terminado cuando funciona, está probado y otro integrante lo revisó. El cronograma está en la sección 7.2.

Riesgos: si el Sprint 2 se retrasa, RF-05 (clientes) y la anulación pasan al Sprint 3 sin afectar la venta básica. El plan gratuito de Supabase no hace copias automáticas: una tarea programada exporta la base cada día (RNF-06).

4.6 Consolidar la propuesta

- **Integración:** Int. 1 unió las secciones en este único documento y revisó el formato.
- **Coherencia:** cada objetivo tiene sus requisitos y su sprint; por ejemplo, OE-01 (punto de venta) → RF-06 y RF-07 → Sprint 2 (A7).
- **Participación:** cada integrante redactó sus secciones (ver sección 6) y los tres revisaron el documento completo.

5. Entregables obligatorios

El documento contiene las 11 secciones mínimas que pide la guía:

N.º	Sección obligatoria	Dónde está en este documento
1	Portada	Encabezado de la página 1: proyecto, curso, integrantes y fecha
2	Descripción del proyecto	Sección 1: nombre, contexto, descripción general, arquitectura y usuarios
3	Planteamiento del problema	Secciones 3 y 4.1: situación actual, causas, consecuencias y necesidad
4	Justificación	Sección 4.2: importancia, beneficios y razones de la solución web
5	Objetivos	Sección 4.3: 1 objetivo general y 5 específicos (OE-01 a OE-05)
6	Alcance	Sección 4.3: 8 módulos, exclusiones y usuarios principales
7	Requisitos funcionales	Sección 4.4: 9 requisitos con código RF-01 a RF-09
8	Requisitos no funcionales	Sección 4.4: 7 requisitos con código RNF-01 a RNF-07

9	Plan de trabajo	Sección 4.5: actividades, responsables, secuencia y duración
10	Cronograma	Sección 7.2: tabla por actividades, semanas y responsables
11	Conclusiones	Sección 11: síntesis, utilidad y viabilidad preliminar

6. Organización del equipo

Los 3 integrantes cubren los 5 roles de la guía (dos asumen roles compatibles). Todos conocen el documento completo y pueden sustentarlo.

Integrante	Roles de la guía	Secciones que redacta	Rol en la construcción
Int. 1	Coordinador del proyecto + Responsable de documentación	1, 2, 4.6, 5, 6, 11 e integración final	Líder técnico y desarrollador de la API REST
Int. 2	Analista del problema + Responsable de planificación	3, 4.1, 4.2, 4.5 y 7	Desarrollador del frontend
Int. 3	Analista de requisitos	4.3, 4.4, 8, 9 y 10	Base de datos PostgreSQL, reportes y pruebas (QA)

7. Cronograma de trabajo sugerido

7.1 Jornada de elaboración de la propuesta

Etapa	Actividad	Tiempo	Responsable
1	Organización del equipo y selección del proyecto	30 min	Int. 1 (todos)
2	Planteamiento del problema y justificación	60 min	Int. 2
3	Definición de objetivos, usuarios y alcance	45 min	Int. 3
4	Elaboración de requisitos	90 min	Int. 3
5	Plan de trabajo y cronograma	45 min	Int. 2
6	Integración, revisión y entrega	60 min	Int. 1 (todos revisan)
	Total	**5 h 30 min**	

7.2 Cronograma del desarrollo del sistema (10 semanas)

3 semanas de análisis y diseño, 5 de desarrollo y 2 de pruebas, despliegue y cierre.

ID	Actividad	Resp.	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
A0	Coordinación y seguimiento	Int. 1	■	■	■	■	■	■	■	■	■	■
A1	Entrevista al negocio piloto	Int. 2	■									
A2	Requisitos e historias de usuario	Int. 3	■	■								
A3	Modelo de datos (E-R)	Int. 3		■	■							
A4	Prototipos de pantallas	Int. 2		■	■							
A5	Contrato API y entorno	Int. 1			■							
A6	Sprint 1: acceso y productos	Todos				■	■					

A7	Sprint 2: inventario y ventas	Todos						■	■			
A8	Sprint 3: caja y reportes	Todos								■		
A9	Pruebas y correcciones	Int. 3									■	
A10	Prueba de aceptación	Int. 2									■	
A11	Despliegue	Int. 1										■
A12	Capacitación y manual	Int. 2										■
A13	Documentación y presentación	Int. 1										■

Hitos de control:

1. **Fin de S3:** requisitos, modelo de datos, contrato de la API y prototipos aprobados.
2. **Fin de S5:** primer módulo funcionando de punta a punta (acceso, configuración y productos).
3. **Fin de S7:** primera venta completa registrada.
4. **Fin de S9:** sistema completo, probado y aceptado por el negocio piloto.
5. **Fin de S10:** sistema publicado, usuarios capacitados y proyecto entregado.

8. Condiciones de presentación

Condición de la guía	**Cómo la cumple Ranqhay**
Entrega grupal en el plazo del docente	Un único documento del equipo de 3 integrantes
Estructura ordenada, redacción clara y terminología técnica	Sigue los 11 apartados de la guía; usa términos como API REST, JWT, transacción, sprint
Condición de la guía	**Cómo la cumple Ranqhay**
Requisitos relacionados con problema, objetivos y alcance	Cada RF pertenece a un módulo del alcance y a un objetivo específico
Cronograma realista y relacionado con las actividades	Las 14 actividades del plan aparecen en el cronograma, con 5 hitos de control
No se exige código ni sistema funcional	La propuesta es conceptual; el anexo solo orienta la etapa posterior
El equipo puede sustentar sus decisiones	Cada integrante redacta y sustenta sus secciones; todos revisan el conjunto

9. Criterios y ponderación

Ubicación de la evidencia de cada criterio de la rúbrica:

N.º	**Criterio**	**Puntaje**	**Peso**	**Dónde se evidencia**
1	Planteamiento del problema	4	20 %	Secciones 3 y 4.1
2	Justificación del proyecto	3	15 %	Sección 4.2
3	Objetivos y definición del alcance	3	15 %	Sección 4.3
4	Requisitos funcionales y no funcionales	4	20 %	Sección 4.4
5	Plan de trabajo y cronograma	3	15 %	Secciones 4.5 y 7.2

6	Organización y calidad del documento	2	10 %	Secciones 1 a 11 y sección 5
7	Coherencia e integración de la propuesta	1	5 %	Sección 4.6
	Total	**20**	**100 %**	

10. Descriptores por nivel de desempeño

Autoevaluación del equipo frente al nivel **Excelente** de cada criterio:

Criterio	Lo que pide «Excelente»	Evidencia en Ranqhay
1. Problema	Problemática concreta, afectados, causas, consecuencias y contexto delimitado	Control manual y disperso; 6 causas con consecuencias; dueño, cajeros y clientes; comercio de un local con venta presencial
2. Justificación	Beneficios relacionados con el problema y las necesidades de los usuarios	Tabla problema → beneficio → cómo lo logra; 5 razones para elegir una solución web
3. Objetivos y alcance	Objetivo general, 2 o más específicos; funcionalidades, usuarios y exclusiones	1 general y 5 específicos; 8 módulos, 2 usuarios y 6 exclusiones
4. Requisitos	6 o más RF y 4 o más RNF, claros, verificables y coherentes	9 RF y 7 RNF con código; cada RNF indica cómo se verifica
5. Plan y cronograma	Actividades ordenadas, responsables, duraciones y cronograma coherente	14 actividades con dependencias; Gantt de 10 semanas y 5 hitos
6. Documento	Completo, estructurado, legible y con todos los apartados	Los 11 apartados obligatorios presentes (sección 5)
7. Coherencia	Correspondencia entre problema, objetivos, alcance, requisitos y plan	Cada problema → objetivo → RF → sprint (sección 4.6)

11. Resultado esperado

Pregunta de la guía	Respuesta de Ranqhay
¿Qué problema busca resolver?	El control manual y disperso de ventas, stock y caja en pequeños negocios, que causa descuadres, agotados, errores de precio y decisiones sin datos.
¿Por qué se necesita una solución tecnológica?	Porque solo un registro único e inmediato actualiza a la vez stock, caja y reportes; la web permite usarlo desde cualquier equipo, sin instalar nada.
¿Quiénes lo usarían y qué necesitan?	El administrador (control, datos para decidir) y el cajero (vender rápido y sin errores).
¿Qué funciones y calidad ofrecerá?	8 módulos (RF-01 a RF-09) con seguridad por roles, HTTPS, venta en ≤ 2 s, transacciones y copia diaria (RNF-01 a RNF-07).
¿Cómo se planifica su desarrollo?	10 semanas en 6 fases y 3 sprints, con responsables por actividad y 5 hitos de control.

Conclusiones

Ranqhay ataca la causa del problema: registrar cada venta al momento en un sistema único. Es útil para el dueño (control y datos), los cajeros (rapidez y menos errores) y los clientes (mejor atención). **Viabilidad preliminar:**

- **Técnica:** React, API REST en Node.js y PostgreSQL, tecnologías libres del curso; se excluyen la facturación electrónica y los pagos en línea.
- **Económica:** sin licencias; hosting propio de HostArmada y plan gratuito de Supabase.
- **Operativa:** basta saber usar un navegador; meta de 30 minutos de capacitación.
- **De tiempo:** 10 semanas en 3 sprints con entregas parciales; lo de prioridad alta queda listo aunque haya retrasos.

La propuesta es el punto de partida para el análisis, diseño y desarrollo; el anexo resume cómo empezar a construirla.

Anexo. Guía para pasar a la construcción

Traduce la propuesta a decisiones técnicas iniciales para empezar a programar. La arquitectura (frontend web, API REST y PostgreSQL) es fija; las librerías pueden ajustarse en la fase de diseño.

Stack tecnológico propuesto

Capa	Tecnología	Por qué
Frontend web	React 18 + Vite + TypeScript, React Router, Zustand	Tecnologías que el equipo ya usa; consume la API REST con fetch
Estilos	Tailwind CSS	Pantallas adaptables al celular (RNF-04)
API REST	Node.js + Express + TypeScript	Endpoints JSON por módulo (RNF-07); el mismo lenguaje en frontend y backend
Base de datos	PostgreSQL en Supabase + Prisma ORM	Relacional, con claves foráneas y transacciones (RNF-05); servicio administrado
Seguridad	JWT en el encabezado Authorization, bcrypt, CORS solo para el dominio del frontend	RNF-01
Validación	Zod en la API	Rechaza datos inválidos con código 400 antes de tocar la base de datos

Documentación y pruebas	Swagger (OpenAPI) y Postman	RNF-07
Tickets y reportes	pdfkit o jsPDF (PDF), exceljs (Excel)	RF-07 y RF-09
Versionado	Git + GitHub (repositorio público sankarea270/Ranqhay)	RNF-07
Despliegue	HostArmada con dominio propio: frontend como archivos estáticos, API como aplicación Node.js; base de datos en Supabase	Hosting que el equipo ya tiene, con SSL gratuito (RNF-01); Supabase en plan gratuito

Si el curso pide otro lenguaje para la API (por ejemplo, Java con Spring Boot), solo cambia esa fila: la API seguirá siendo REST y la base de datos, PostgreSQL.

Modelo de datos preliminar (9 entidades)

Entidad	Atributos principales	Relación
usuario	id, nombre, usuario, contrasena_hash, rol (ADMIN o CAJERO), activo	Registra ventas, cajas y entradas
categoria	id, nombre	Agrupa muchos productos
producto	id, codigo, codigo_barras, nombre, categoria_id, unidad_medida, precio_compra, precio_venta, stock, stock_minimo, activo	Pertenece a una categoría
entrada_inventario	id, producto_id, usuario_id, proveedor (texto), cantidad, costo_unitario, fecha	Aumenta el stock de un producto
cliente	id, tipo_documento (DNI o RUC), numero_documento, nombre, telefono	Tiene muchas ventas
caja	id, usuario_id, monto_inicial, monto_esperado, monto_contado, diferencia, apertura, cierre, estado	Agrupa las ventas de un turno
venta	id, numero_ticket, fecha, cliente_id, usuario_id, caja_id, subtotal, descuento, total, metodo_pago, monto_recibido, vuelto, estado (PAGADA o ANULADA), motivo_anulacion	Tiene muchos detalles
detalle_venta	id, venta_id, producto_id, cantidad, precio_unitario, subtotal	Une cada venta con sus productos
Entidad	**Atributos principales**	**Relación**
negocio	id, nombre_comercial, ruc, direccion, telefono, logo_url, moneda, porcentaje_impuesto, unidades_medida	Un solo registro con los datos de tickets y reportes (RF-02)

El detalle guarda el precio unitario del momento de la venta: si luego cambia el precio del producto, los tickets y reportes antiguos no se alteran.

Endpoints principales de la API

Módulo	Endpoints	Requisitos
Acceso	`POST /api/auth/login·GET, POST, PUT /api/usuarios`	RF-01
Configuración	`GET, PUT /api/negocio`	RF-02

Productos	`GET, POST, PUT /api/productos` · `GET /api/productos?buscar=` · `GET, POST /api/categorias`	RF-03
Inventario	`POST /api/entradas` · `GET /api/productos/stock-bajo`	RF-04
Clientes	`GET, POST /api/clientes`	RF-05
Ventas	`POST /api/ventas` · `GET /api/ventas/:id/ticket` · `POST /api/ventas/:id/anular`	RF-06, RF-07
Caja	`POST /api/caja/abrir` · `POST /api/caja/cerrar` · `GET /api/caja/actual`	RF-08
Reportes	`GET /api/ventas?desde=&hasta=&cajero=` · `GET /api/reportes/ventas` · `GET /api/reportes/exportar` · `GET /api/dashboard`	RF-09

Convenciones de la API REST

- **Formato:** JSON; fechas en ISO 8601 y montos con 2 decimales.
- **Autenticación:** token JWT en el encabezado `Authorization: Bearer <token>` en todas las rutas, excepto `/api/auth/login`.
- **Códigos:** 200 (correcto), 201 (creado), 400 (datos inválidos), 401 (sin sesión), 403 (rol sin permiso), 404 (no existe) y 409 (conflicto, p. ej. stock insuficiente).
- **Documentación:** Swagger en `/api/docs` (RNF-07).

Ejemplo: el frontend registra una venta (RF-06) con `POST /api/ventas` y este cuerpo JSON:

```
{
  "clienteId": 12,
  "metodoPago": "EFECTIVO",
  "montoRecibido": 50.00,
  "items": [
    { "productoId": 8, "cantidad": 2 },
    { "productoId": 15, "cantidad": 1 }
  ]
}
```

La API valida el stock, guarda la venta y su detalle en una sola transacción de PostgreSQL y responde 201 con el número de ticket, el total y el vuelto. Si un producto no tiene stock suficiente, responde 409 y no guarda nada.

Despliegue en HostArmada y Supabase

Frontend y API en dos subdominios del dominio propio, cada uno con SSL gratuito; la API solo acepta peticiones del subdominio del frontend (CORS). Solo la API se conecta a Supabase.

1. En el cPanel de HostArmada, crear `ventas.tudominio.com` (frontend) y `api.tudominio.com` (API) y activar el SSL gratuito en ambos.
2. En Supabase, crear el proyecto en la región más cercana a Lima (São Paulo), crear un usuario `prisma` con permisos sobre el esquema `public` y copiar la cadena de conexión del pooler Supavisor en modo sesión (puerto 5432), que funciona por IPv4 desde el hosting.
3. En Supabase, desactivar la Data API («Enable Data API») para que nadie lea las tablas sin pasar por la API propia (RNF-07).
4. API: subir el backend con Git de cPanel o por SSH; crear la aplicación en Node.js Selector apuntando a `api.tudominio.com`; definir `DATABASE_URL` y `JWT_SECRET`; ejecutar `npm install`, las migraciones de Prisma y el seed.
5. Frontend: compilar con `npm run build` usando `VITE_API_URL=https://api.tudominio.com/api` y subir `dist` a la carpeta del subdominio `ventas`, con un `.htaccess` que dirija todas las rutas a `index.html`.

6. Copias diarias (RNF-06): programar una exportación diaria con `supabase db dump` (por ejemplo, con GitHub Actions) y conservar las de los últimos 7 días.
7. Probar en producción: abrir `https://ventas.tudominio.com`, iniciar sesión, registrar una venta y revisar `https://api.tudominio.com/api/docs`.

Fuentes, consultadas en octubre de 2026: Development Hosting de HostArmada (Node.js en planes compartidos, SSH y SSL gratuito); precios de Supabase (plan gratuito: 500 MB por proyecto, pausa tras 1 semana sin actividad, sin copias automáticas); conexión a Postgres en Supabase (pooler por IPv4, puertos 5432 y 6543); Prisma con Supabase; desactivar la Data API; copias de seguridad en Supabase.

Estructura de carpetas sugerida

`Ranqhay/`	github.com/sankarea270/Ranqhay
`frontend/`	React + Vite + TypeScript
`src/pages/`	Login, Venta, Productos, Inventario, Caja, Reportes, Panel
`src/components/`	Carrito, BuscadorProducto, Ticket, Tabla, Modal
`src/services/`	Llamadas a la API REST
`src/store/`	Sesión y carrito (Zustand)
`.env.example`	VITE_API_URL
`backend/`	Node + Express + TypeScript
`src/routes/`	Una ruta por módulo
`src/controllers/`	Reciben la petición y responden
`src/services/`	Lógica: venta con transacción, cierre de caja
`src/middlewares/`	Autenticación JWT y control de roles
`prisma/`	schema.prisma, migraciones y seed.ts
`.env.example`	DATABASE_URL y JWT_SECRET, sin valores reales
`.gitignore`	Incluye .env y node_modules
`README.md`	Instalación y uso (RNF-07)

Primeros pasos al iniciar la construcción

1. Usar el repositorio público sankarea270/Ranqhay e invitar a los 3 integrantes; trabajar con una rama por requisito (p. ej. `feature/RF-06-venta`) y unirla con pull request revisado por otro integrante.
2. Como el repositorio es público, el primer commit incluye un `.gitignore` con `.env` y `node_modules`, y solo archivos `.env.example` sin valores reales: la cadena de Supabase y el `JWT_SECRET` nunca se suben a GitHub.
3. Crear un tablero (GitHub Projects o Trello) con una tarjeta por requisito y su prioridad.
4. Int. 3: crear el proyecto en Supabase y cargar con Prisma el esquema de las 9 entidades y un seed con un administrador, un cajero y 20 productos de prueba.
5. Int. 1: levantar la API REST base con el login, el JWT, el middleware de roles y Swagger.
6. Int. 2: crear el proyecto React con la pantalla de login conectada a `POST /api/auth/login` y el menú que cambia según el rol.
7. Al cerrar cada sprint, probar sus requisitos con la columna «Cómo se verifica» y el ejemplo de la sección 4.4.